

HDOC-18

Question:

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer.

- Check whether it is a Strong number.
- Check whether it is an Armstrong number.
- Check whether it is a Harshad number.
- Check whether it is a Duck number.

Task1: Algorithm:

Main Algorithm

Step 1: Start.

Step 2: Declare choice as an integer.

Step 3: Display the menu:

- 1 → Strong Number
- 2 → Armstrong Number
- 3 → Harshad Number
- 4 → Duck Number

Step 4: Read choice.

Step 5: Use switch(choice):

- If choice = 1, call strongNumber()
- If choice = 2, call armstrongNumber()
- If choice = 3, call harshadNumber()
- If choice = 4, call duckNumber()
- Otherwise, display "Invalid choice"

Step 6: Stop.

Algorithm for Strong Number

A number is Strong if the sum of factorials of its digits equals the original number.

Example:

$$145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$$

Steps:

1. Input num.
2. Store num in temp.
3. Initialize sum = 0.
4. Repeat while temp > 0:
 - Find digit using digit = temp % 10.
 - Set fact = 1.
 - Calculate factorial of digit using a for loop.

- Add factorial to sum.
 - Remove last digit using $\text{temp} = \text{temp} / 10$.
5. If $\text{sum} == \text{num}$, display Strong number.
 6. Otherwise, display Not a Strong number.

Algorithm for Armstrong Number

An Armstrong number is a number equal to the sum of its digits raised to the power of the number of digits.

Example:

$$153 = 1^3 + 5^3 + 3^3 = 153$$

Steps:

1. Input num.
2. Store num in temp.
3. Count the number of digits.
4. Again set $\text{temp} = \text{num}$.
5. Initialize $\text{sum} = 0$.
6. Extract each digit.
7. Calculate $\text{digit}^{\text{digits}}$ using a loop.
8. Add it to sum.
9. Remove the last digit.
10. If $\text{sum} == \text{num}$, display Armstrong number.
11. Otherwise, display Not an Armstrong number.

Algorithm for Harshad Number

A Harshad number is divisible by the sum of its digits.

Example:

$$18 \rightarrow 1 + 8 = 9$$

$$18 \% 9 = 0$$

Therefore, 18 is a Harshad number.

Steps:

1. Input num.
2. Store it in temp.
3. Initialize $\text{sum} = 0$.
4. Extract every digit and add it to sum.
5. Check $\text{num} \% \text{sum}$.
6. If the remainder is 0, display Harshad number.
7. Otherwise, display Not a Harshad number.

Algorithm for Duck Number

A positive integer is a Duck number if it contains at least one zero in its digits.

Example: 1023 is a Duck number.

Steps:

1. Input num.
2. Store it in temp.
3. Set $\text{flag} = 0$.
4. Extract every digit.
5. If any digit is 0, set $\text{flag} = 1$.
6. If $\text{flag} == 1$, display Duck number.
7. Otherwise, display Not a Duck number.

Task2: Test case table:

Test Case	Choice	Input Number	Expected Output	Actual Output	Result
1	1	145	145 is a Strong number	145 is a Strong number	PASS
2	1	123	123 is not a Strong number	123 is not a Strong number	PASS
3	2	153	153 is an Armstrong number	153 is an Armstrong number	PASS
4	2	125	125 is not an Armstrong number	125 is not an Armstrong number	PASS
5	3	18	18 is a Harshad number	18 is a Harshad number	PASS
6	3	19	19 is not a Harshad number	19 is not a Harshad number	PASS
7	4	1023	1023 is a Duck number	1023 is a Duck number	PASS
8	4	1234	1234 is not a Duck number	1234 is not a Duck number	PASS
9	5	—	Invalid choice	Invalid choice	PASS

Task3: C program:

```
#include <stdio.h>
void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();
int main()
{
    int choice;
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");
```

```

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        strongNumber();
        break;
    case 2:
        armstrongNumber();
        break;
    case 3:
        harshadNumber();
        break;
    case 4:
        duckNumber();
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}
/* Function to check Strong Number */
void strongNumber()
{
    int num, temp, digit;
    int sum = 0, fact, i;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;
    while(temp > 0)
    {
        digit = temp % 10;
        fact = 1;
        for(i = 1; i <= digit; i++)
        {
            fact = fact * i;
        }
        sum = sum + fact;
        temp = temp / 10;
    }
    if(sum == num)
        printf("%d is a Strong number.\n", num);
    else
        printf("%d is not a Strong number.\n", num);
}

```

```

}
/* Function to check Armstrong Number */
void armstrongNumber()
{
    int num, temp, digit;
    int digits = 0, sum = 0;
    int power, i;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    temp = num;
    while(temp > 0)
    {
        digits++;
        temp = temp / 10;
    }
    temp = num;
    while(temp > 0)
    {
        digit = temp % 10;
        power = 1;
        for(i = 1; i <= digits; i++)
        {
            power = power * digit;
        }
        sum = sum + power;
        temp = temp / 10;
    }
    if(sum == num)
        printf("%d is an Armstrong number.\n", num);
    else
        printf("%d is not an Armstrong number.\n", num);
}

/* Function to check Harshad Number */
void harshadNumber()
{
    int num, temp, digit;
    int sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while(temp > 0)
    {
        digit = temp % 10;
        sum = sum + digit;
    }
}

```

```

        temp = temp / 10;
    }

    if(num % sum == 0)
        printf("%d is a Harshad number.\n", num);
    else
        printf("%d is not a Harshad number.\n", num);
}

/* Function to check Duck Number */
void duckNumber()
{
    int num, temp, digit;
    int flag = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &num);
    temp = num;

    while(temp > 0)
    {
        digit = temp % 10;
        if(digit == 0)
        {
            flag = 1;
            break;
        }
        temp = temp / 10;
    }
    if(flag == 1)
        printf("%d is a Duck number.\n", num);
    else
        printf("%d is not a Duck number.\n", num);
}

```

Task4: Output:

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive interger: 145
145 is a Strong number.
```

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive interger: 123
123 is not a Strong number.
```

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.
```

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 125
125 is not an Armstrong number.
```

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.
```

```
(kali㉿kali)-[~/hdoc18.c]
└─$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 19
19 is not a Harshad number.
```

```
(kali㉿kali)-[~/hdoc18.c]
$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1023
1023 is a Duck number.
```

```
(kali㉿kali)-[~/hdoc18.c]
$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1234
1234 is a not a Duck number.
```

```
(kali㉿kali)-[~/hdoc18.c]
$ ./a.out
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 5
Invalid choice
```